# A General Variable Neighborhood Search for Logistic Network Flow Design with Nonlinear Edge Costs and Transshipment Capacities

Varvara Kovaleva[1] and Alexander Ponomarenko[2]

[1] HSE University, Nizhny Novgorod, Russia
varvara_kovaleva03@mail.ru
[2] Laboratory of Algorithms and Technologies for Network Analysis, HSE University, Nizhny Novgorod, Russia
aponomarenko@hse.ru

**Abstract.** We study how to route multiple commodities through a logistic transport network so as to minimise the joint cost of vehicle movements and of transshipment (reload) operations at intermediate storages. The problem combines two features common in practice but rarely treated together: edge costs that are *nonlinear* in the transported volume, because freight is carried by vehicles of fixed capacity, and *hard upper bounds* on the volume that can be reloaded at each node. We formulate it as a mixed integer linear program in both an arc-based and a path-based form, derive its linear relaxation, and propose a solver based on General Variable Neighborhood Search (GVNS). The method works in the path space and relies on three problem-specific neighborhoods that generate candidate paths by (i) shortest paths under a reload-aware metric, (ii) edges carrying partially loaded vehicles, and (iii) detours around congested storages. We evaluate it on our open benchmark [24]: realistic logistic networks calibrated on real road graphs of nine European countries, five U.S. states and the European part of Russia. On 35 synthetic and 15 real-geometry instances with up to 150 storages, GVNS returns solutions within about one percent of the best dual bound on small networks—a branch-and-bound lower bound substantially tighter than the LP relaxation—while on the medium instances a time-limited exact solver given a comparable wall-clock budget returns solutions up to 55% more expensive than those of GVNS.

**Keywords:** multi-commodity flow · network design · variable neighborhood search · metaheuristics · mixed integer programming · logistics

## 1 Introduction

Organising commodity flows between distribution centres is a central problem of logistic networks, and the scale of modern warehousing keeps demand for efficient planning algorithms high. Two features make realistic instances hard.

First, the cost of using a road is *nonlinear* in the volume carried: goods travel in vehicles of fixed capacity, so the billed quantity is the integer *number of vehicles*, obtained by rounding the volume up to that capacity. Second, every storage has a limited *transshipment capacity*—an upper bound on the volume that may be unloaded, stored and reloaded there. Together they turn an otherwise classical multi-commodity flow problem into a mixed integer program with node capacities and a step-wise vehicle cost term.

*Related work.* The problem belongs to the family of capacitated multicommodity network design (MCND) [14,7], which is NP-hard. Exact approaches solve path-based formulations by column generation and branch-and-price [3]; at practical scale heuristics dominate, notably cycle-based neighborhoods inside tabu search [15], local branching [30] and matheuristics that call a MILP solver on restricted subproblems [18]. Metaheuristics for multi-commodity flow more broadly are surveyed in [8]. Cost functions of the kind studied here appear as *step* or discrete costs: Gabrel et al. [11] give an exact Benders-type procedure for general step costs, and Croxton et al. [9] compare MILP models of nonconvex piecewise-linear costs. Capacity on the *node* side is instead typical of hub location, where hub throughput is bounded [1]. Variable Neighborhood Search [28,16,17] has proven effective across a broad range of optimisation problems: Masri et al. [26] applied a multi-start VNS to the single-path multicommodity flow problem, General VNS performs well on capacitated vehicle routing and other flow-constrained logistic problems [2], and problem-specific heuristics do well on real-life logistic routing, such as the truck-and-trailer routing problem with time windows and split deliveries [5,6].

We are not aware of an existing solver for this variant. Published MCND codes charge a *fixed* cost per open arc rather than a per-vehicle step cost, and they do not bound the volume transshipped at a node, so results obtained with them would not be comparable. The instance corpus we published earlier [24,25] contains networks and demands but no solutions, so there are no published objective values for these instances either. We therefore compare GVNS with bounds and general-purpose solvers on the same model.

*Contributions.* We state the problem and give equivalent arc-based and path-based MILP models with their linear relaxation (Section 2). We describe a GVNS that works in the space of commodity paths, with three neighborhoods based on reload costs, partial vehicle loads and storage congestion (Section 3). We report a computational study on that corpus (Section 4), comparing GVNS with the LP relaxation, its rounded version, and the dual bounds and incumbents of an exact solver (Section 5), and an analysis of how much each neighborhood and the VND contribute.

## 2 Problem statement and model

### 2.1 Problem description

Consider a directed graph $G = (V, E)$ whose nodes are storages and whose edges are roads. Each storage $v \in V$ has a per-unit reload cost $f_v \in \mathbb{R}_+$ and a hard *transshipment capacity* $W_v \in \mathbb{R}_+$, the largest volume that may be unloaded, stored and reloaded there; each edge $e \in E$ has a per-vehicle traversal cost $c_e \in \mathbb{R}_+$. Freight moves in vehicles of capacity $C \in \mathbb{R}_+$, so what an edge actually costs is set by the *integer* number of vehicles it carries. We are given a set $K$ of transportation requests (commodities); request $k \in K$ must ship a demand $d_k \in \mathbb{R}_+$ from an origin $s_k \in V$ to a destination $t_k \in V$.

The problem is to route all requests so as to minimise the total vehicle movement cost plus the total transit reload cost, subject to demand satisfaction, edge vehicle capacities and node transshipment capacities. It is thus a capacitated multicommodity network design problem with step edge costs and node transshipment capacities, and it admits two equivalent formulations, arc-based and path-based, given next. Capacities and costs are treated as constants; on real networks they vary with the time of day, a point we return to in Section 6.

### 2.2 Mathematical model

**Decision variables.** *Arc-based:* let $x_e^k \in \mathbb{R}_+$ be the flow of commodity $k \in K$ on edge $e \in E$ and $y_e \in \mathbb{Z}_+$ the number of vehicles traversing $e$; the model has $|K| \cdot |E| + |E|$ variables. *Path-based:* let $\Pi_k$ be the set of all paths from $s_k$ to $t_k$, let $x_p^k \in \mathbb{R}_+$ be the flow of commodity $k$ on path $p \in \Pi_k$, and $y_e \in \mathbb{Z}_+$ as above; the model has $\sum_{k \in K} |\Pi_k| + |E|$ variables.

**Objective.** With $\delta^-(v) \subseteq E$ and $\delta^+(v) \subseteq E$ the sets of edges entering and leaving $v$, minimise vehicle movement cost (first term) plus transit reload cost (second term).

*Arc-based:*

$$\min \left( \sum_{e \in E} c_e \, y_e \; + \sum_{k \in K} \sum_{\substack{v \in V \\ v \neq s_k,\, v \neq t_k}} \sum_{e \in \delta^-(v)} f_v \, x_e^k \right). \tag{1}$$

*Path-based:*

$$\min \left( \sum_{e \in E} c_e \, y_e \; + \sum_{k \in K} \sum_{p \in \Pi_k} \sum_{\substack{v \in p \\ v \neq s_k,\, v \neq t_k}} f_v \, x_p^k \right). \tag{2}$$

**Constraints.** *(1) Demand satisfaction.*

$$\text{arc:} \quad \sum_{e \in \delta^+(v)} x_e^k - \sum_{e \in \delta^-(v)} x_e^k = \begin{cases} d_k, & v = s_k \\ -d_k, & v = t_k \\ 0, & \text{otherwise} \end{cases} \qquad \forall k \in K,\ \forall v \in V; \tag{3}$$

$$\text{path:} \quad \sum_{p \in \Pi_k} x_p^k = d_k, \qquad \forall k \in K. \tag{4}$$

*(2) Vehicle capacity on edges.* Enough vehicles must run on each edge to carry the total flow:

$$\text{arc:} \quad \sum_{k \in K} x_e^k \le C\, y_e, \qquad \forall e \in E; \tag{5}$$

$$\text{path:} \quad \sum_{k \in K} \sum_{p \in \Pi_k \colon e \in p} x_p^k \le C\, y_e, \qquad \forall e \in E. \tag{6}$$

*(3) Storage transshipment capacity.*

$$\text{arc:} \quad \sum_{\substack{k \in K \\ v \ne s_k,\, v \ne t_k}} \sum_{e \in \delta^-(v)} x_e^k \le W_v, \qquad \forall v \in V; \tag{7}$$

$$\text{path:} \quad \sum_{\substack{k \in K \\ v \ne s_k,\, v \ne t_k}} \sum_{\substack{p \in \Pi_k \\ v \in p}} x_p^k \le W_v, \qquad \forall v \in V. \tag{8}$$

### 2.3 Complexity and linear relaxation

The objective and all constraints are linear and the variables are mixed—the vehicle counts $y_e$ are integer while the flows $x_e^k$ (or $x_p^k$) are continuous—so the problem is a *mixed integer linear program* (MILP), an NP-hard class for which no polynomial-time exact algorithm is known in general.

A standard way to bound the optimum is the LP relaxation: dropping the integrality of $y_e$ turns the MILP into a linear program solvable in polynomial time, and its optimum is a lower bound (for minimisation). Once $y_e$ is continuous, the capacity constraints (5)–(6) hold with equality at optimum, so $y_e$ can be eliminated:

$$y_e = \frac{1}{C} \sum_{k \in K} x_e^k \quad \text{(arc)}, \qquad y_e = \frac{1}{C} \sum_{k \in K} \sum_{p \in \Pi_k \colon e \in p} x_p^k \quad \text{(path)}. \tag{9}$$

Substituting (9) into the objective yields a relaxed problem in the flow variables only. In the path-based form,

$$\min \sum_{k \in K} \sum_{p \in \Pi_k} x_p^k \left( \frac{1}{C} \sum_{e \in p} c_e + \sum_{\substack{v \in p \\ v \ne s_k,\, v \ne t_k}} f_v \right), \tag{10}$$

which makes explicit that an optimal relaxed solution prefers paths with low travel cost and few transit nodes.

## 3 The GVNS solution approach

### 3.1 General Variable Neighborhood Search

VNS [16,17] is a metaheuristic whose core idea is the systematic change of neighborhood structures while exploring the solution space. Given a problem

$\min\{f(x) : x \in X\}$ and a family of neighborhoods $N_k(x),\ k = 1, \ldots, k_{\max}$, the classical scheme alternates three steps: a *shake* (a random move in $N_k$ to escape local optima), a *local search*, and a *neighborhood change* (on improvement, accept the move and reset $k \leftarrow 1$; otherwise advance to $k+1$). When the local search is itself a Variable Neighborhood Descent (VND)—a deterministic sweep through the neighborhoods ordered from intensification to diversification—the method is called General VNS (GVNS), which often outperforms single-neighborhood heuristics [21,27]. Our instantiation searches the space of commodity paths; its concrete form is Algorithm 2, built on the neighborhood-processing primitive of Algorithm 1.

### 3.2 Preprocessing and initial solution

Transport networks are typically sparse directed graphs in which not every pair of storages is joined by a direct road, and not every transit node needs a reload. Working on the raw graph both shrinks the feasible region (through node capacities) and may even render the instance infeasible if no path exists between an origin and a destination.

To avoid this, the graph is *completed*: for every pair of storages $v, u$ we add (or update) an edge whose cost equals that of the shortest path from $v$ to $u$ in $G$. Each edge of the resulting complete graph thus represents the cheapest stop-free route between its endpoints, and $c_{vu}$ denotes its per-vehicle cost. From here on $G = (V, E)$ denotes this completed graph, so every later formula ranges over all ordered pairs of storages. This guarantees the existence of a feasible solution, because flow routed on the direct origin–destination edge does not pass through any transit node and hence violates no node-capacity constraint. From the relaxed objective (10) it follows that the cheapest, transit-free direct edges are exactly the ones a good solution prefers. We therefore take as the initial solution the relaxed LP solution that sends each commodity entirely along its direct origin–destination edge, with vehicle counts rounded up:

$$y_e = \left\lceil \tfrac{1}{C} \sum_{k \in K} x_e^k \right\rceil, \qquad \forall e \in E. \tag{11}$$

### 3.3 Neighborhoods

Since the vehicle counts $y_e$ are determined directly from the flows via (11), the neighborhoods vary the flows. Predicting feasible per-edge flows is hard because edges are shared by many commodities and adjacent flows are strongly coupled, so the whole network must be considered at once. A single path, by contrast, describes the routing of just one commodity, and adding a new path to an already feasible solution never worsens it. The neighborhoods therefore *build and modify the set of candidate paths for each commodity*: for every commodity a set of new paths is generated and added to the model, a MILP solver is run for a limited time to redistribute flow, and—if the objective improves—all new paths carrying nonzero flow are kept. Three neighborhoods define how the new paths are generated.

*N1 – Shortest paths (reload-aware).* Classical shortest-path search ignores reload costs. We instead build an auxiliary complete graph $G'$ whose edge weights measure the per-unit cost of moving one unit of commodity, including reloads at the endpoints:

$$c'_{vu} = f_v + f_u + \frac{c_{vu}}{C}, \qquad \forall (v,u) \in E. \tag{12}$$

Before GVNS starts, a pool of shortest paths under $c'$ is precomputed for each commodity; new paths are then drawn from it, either at random or preferring low-cost paths. During VND all combinations of the two parameters—the number of new paths per commodity and the selection strategy—are swept (one to three paths, both strategies); in Shake, five random paths are always added, which helps escape local optima.

*N2 – Partially loaded edges.* A partially loaded vehicle has spare room: routing extra flow over such an edge can free another vehicle and reduce the vehicle count. For every edge we compute, for every $e \in E$, the residual load $load_e = \left(\sum_{k\in K} \sum_{p\in \Pi_k:\, e\in p} x_p^k\right) \bmod C$, and build a graph $G''$ that favours edges with free capacity. Fully loaded edges ($load_e = 0$) are least attractive, since new flow would require an extra vehicle, and keep their weight from $G'$. The remaining edges receive

$$c''_{vu} = f_v + f_u + \frac{load_{vu}}{C}\left(\frac{c_{vu}}{C}\right)^{\alpha}, \qquad \alpha \le 1, \tag{13}$$

so that smaller residual load (more free room) lowers the weight and raises priority; $\alpha$ tunes how strongly travel cost influences the weight, and shortest-path search on $G''$ then tends to build paths through partially loaded vehicles. The number of new paths is set dynamically: a counter of fully loaded paths and an accumulator of total free room stop generation once a threshold is reached (in VND, at most two fully loaded paths or free room above one vehicle capacity; in Shake, three paths and $5C$).

*N3 – Detours around congested storages.* Storages whose transshipment volume is at or near their capacity are bottlenecks, and routing around them is desirable. For the current solution we compute, for every $v \in V$, a congestion ratio $overload_v = \frac{1}{W_v} \sum_{k\in K:\, v\neq s_k,t_k} \sum_{p\in \Pi_k:\, v\in p} x_p^k$. Storages are sorted by decreasing $overload_v$ and the single most congested one is deleted from $G'$ (except where it is the commodity's own origin or destination); a shortest path is then recomputed, yielding detours that bypass the busiest node and shift flow to less loaded ones. One new path per commodity is generated in VND and three in Shake. The natural generalisation—removing a whole prefix of $|V|/5$–$|V|/4$ congested storages—was also tried: over all 50 instances and ten seeds it is indistinguishable from the single-node rule (44 of 50 instances within across-seed noise) yet forfeits $N_3$'s benefit on the capacity-tight networks, so we keep the surgical version.

**Algorithm 1:** `ProcessNeighborhood`: enlarge the path pool, re-optimise the flows by MILP, and keep the move iff it improves the objective.

**Function** `ProcessNeighborhood`($P$, $x$, $z$, $N$, *div*, $\tau$)`:`
  `// path pool` $P$`, flows` $x$`, objective` $z = f(x)$`; all updated in place`
  $Q \leftarrow \bigcup_{k \in K}$ `GenPaths`($N$, $k$, *div*) `// candidates;` $N$ `selects the rule`
  **if** $Q = \emptyset$ **then return false**
  $(x', z') \leftarrow$ `SolveMILP`($P \cup Q$, $x$, $\tau$) `// redistribute; warm start` $x$`, limit` $\tau$
  **if** $z' \geq z$ **then** discard $Q$; **return false**
  $P \leftarrow P \cup \{ p \in Q : x'_p > 0 \}$; $x \leftarrow x'$; $z \leftarrow z'$
  prune from $P$ every path carrying zero flow `// keep the model bounded`
  **return true**

### 3.4 Overall algorithm

Algorithm 1 is the primitive shared by the three neighborhoods. Only the generation step `GenPaths` differs between $N_1$, $N_2$ and $N_3$. Algorithm 2 embeds it in the GVNS template: starting from the rounded-LP solution of Eq. (11), each round shakes with the current neighborhood—diversified parameters and a larger MILP budget $\tau_{\text{shk}}$—and then runs the VND sweep $N_1 \to N_2 \to N_3$ with a smaller budget $\tau_{\text{vnd}}$, since VND explores a narrower region. The neighborhood index resets on improvement and advances otherwise; a global limit $t_{\max}$ bounds the run, and the search stops once a full sweep brings no improvement.

## 4 Benchmark instances

We evaluate GVNS on our open benchmark [24,25], published as a Zenodo dataset (CC-BY-4.0); we summarise only what the study needs and refer to [24] for the generator. The corpus targets upper-level road networks connecting regional sorting centres, calibrated on OpenStreetMap [29] data for nine European countries, five large U.S. states and the European part of Russia. It reproduces the structural statistics of real regional transport systems—Zipf city sizes [10], a spatial-network budget topology [12], centrality-driven transshipment capacities [4,22] and a doubly constrained gravity demand model [31]—yielding *near-planar, large-diameter* graphs (mean degree about 3.575), the opposite of small-world networks. The corpus comprises 35 synthetic instances (10 with 10 storages, 7 each with 20 and 30, 5 with 50, and 3 each with 100 and 150) and 15 real-geometry instances built on the calibration road graphs (11–90 storages); we use all 50, each exactly reproducible from its published seed and parameters.

**Algorithm 2:** `GVNS` for logistic network flow design.

**Input :** graph $G = (V, E)$, commodities $K$, vehicle capacity $C$, time limit $t_{\max}$
**Output:** path pool $P$ with commodity flows $x$

build the reload-aware complete graph $G'$; precompute a shortest-path pool per commodity
$P \leftarrow \{\,(s_k, t_k) : k \in K\,\}$; $(x, z) \leftarrow$ `RoundedLP(`$P$`)` `// initial solution, Eq.` (11)
$k \leftarrow 1$
**while** $k \leq 3$ **and** CpuTime $< t_{\max}$ **do**
  `ProcessNeighborhood(`$P,\ x,\ z,\ N_k,\ \mathbf{true},\ \tau_{\text{shk}}$`)` `// shake: diversified, larger budget`
  $z_0 \leftarrow z$
  **repeat**
    $imp \leftarrow$ `ProcessNeighborhood(`$P,\ x,\ z,\ N_1,\ \mathbf{false},\ \tau_{\text{vnd}}$`)`
    **if** $\neg\, imp$ **then** $imp \leftarrow$ `ProcessNeighborhood(`$P,\ x,\ z,\ N_2,\ \mathbf{false},\ \tau_{\text{vnd}}$`)`
    **if** $\neg\, imp$ **then** $imp \leftarrow$ `ProcessNeighborhood(`$P,\ x,\ z,\ N_3,\ \mathbf{false},\ \tau_{\text{vnd}}$`)`
  **until** $\neg\, imp$ **or** CpuTime $\geq t_{\max}$
  **if** $z < z_0$ **then** $k \leftarrow 1$ **else** $k \leftarrow k + 1$ `// neighborhood change`
**return** $(P,\ x)$

## 5 Computational experiments

### 5.1 Setup, baselines and metric

All experiments were run on the HPC cluster "cHARISMa" [23] (nodes with two AMD EPYC CPUs, 44 cores and ≥720 GB RAM, Rocky Linux 9), each job pinned to eight cores. The exact models were solved with HiGHS 1.14.0 [20] and, as an independent cross-check of the lower bounds, with SCIP 10.0.2 [19]; HiGHS is also the metaheuristic's in-loop redistribution solver. GVNS is stochastic (random path choices in the shake step), so every instance was solved with ten independent random seeds and we report the best objective together with its across-seed mean and standard deviation; the component studies of Section 5.3, which compare variants rather than establish the headline numbers, use five seeds each. Unless stated otherwise the exact solves were run to a target MIP gap of zero (i.e. to optimality where possible) under a size-dependent wall-clock budget (30 min for $|V| = 10$, growing to 4 h for $|V| = 150$); GVNS is run with a time limit of twice the number of storages, in minutes. The two budgets are comparable—at $|V| = 50$, for example, GVNS is allowed 100 min against the exact solver's 2 h, and at $|V| = 10$ it is 20 min against 30 min.

The main settings are collected in Table 1. The arc-based MIP grows steeply with $|V|$—from about 6 000 variables at $|V| = 10$ ($|E| = 90$, $|K| = 67$) to $3.7 \times 10^8$ at $|V| = 150$ ($|E| = 22\,350$, $|K| = 16\,762$)—which is why it becomes

**Table 1:** GVNS and exact-solver settings. Values that differ between the VND and the Shake step are given as VND / Shake.

| Setting | Value | Setting | Value |
|---|---|---|---|
| Neighborhoods $k_{\max}$ | 3 | $N_2$ exponent $\alpha$ | 0.5 |
| GVNS limit $t_{\max}$ | $2\|V\|$ min | $N_2$ free-room stop | $1C$ / $5C$ |
| Random seeds | 10 / 5 | $N_3$ storages removed | 1 (worst) |
| Vehicle capacity $C$ | 50 | $N_1$ path pool | 20 |
| $N_1$ paths/commodity | 1–3 / 5 | $N_3$ paths/commodity | 1 / 3 |
| Init. MILP limit | 300 s | Exact gap target | 0 |
| Redistrib. MILP gap | 1% | Exact budget | 30 min–4 h |
| $\tau_{\text{shk}}$ ($N_1$ / rest) | $10+\|V\|$ / $100+2\|V\|$ s | $\tau_{\text{vnd}}$ ($N_1$ / rest) | 25 / 70 s* |

* 15 / 50 s at $|V| = 100$.

intractable well before that size, while GVNS, working in the much smaller path space, remains applicable throughout.

Lacking known optima, we compare GVNS against four references. The *LP relaxation* (Section 2.3) is a lower bound; on the completed graph it is attained by routing every commodity along its direct edge (Section 3), so it is available in closed form at every size, but ignoring vehicle integrality makes it loose. The *rounded LP*—the relaxed solution with $y_e$ rounded up—is the GVNS starting solution and a valid upper bound. The *MIP dual bound* is the global lower bound of the exact solver's search tree, i.e. the LP relaxation strengthened by branching and cutting planes, which rises towards the optimum as the tree is explored; it is a valid lower bound ($\underline{z} \le z^*$) that, unlike the root relaxation, is typically tight (within a few percent of the incumbent), and it is our key reference because it bounds the true optimum even when optimality is not proved. Finally, the *MIP incumbent* is the best feasible solution the exact solver returns within a budget comparable to the GVNS run time.

Because the true optimum is generally not available, the primary quality measure we report is the *optimality gap of GVNS to the best valid lower bound*,

$$gap_{\text{LB}} = \frac{Result_{\text{GVNS}} - \underline{z}}{\underline{z}}, \qquad \underline{z} = \text{best MIP dual bound},$$

which upper-bounds the true optimality gap of GVNS. We additionally report the ratio of each reference objective to that of GVNS ($Ratio_{\text{method}} = Result_{\text{method}}/Result_{\text{GVNS}}$), so values below 1 are better than GVNS and above 1 are worse.

*On the hardness of the exact model.* The arc-based MILP is hard even at the smallest sizes: with the vehicle-count integrality it is a large mixed integer program ($\sim 6000$ variables already at $|V| = 10$), and *no* instance was solved to *proven* optimality within the budgets above by either solver. The branch-and-bound dual bound, however, is tight and well above the LP relaxation, which makes it the right yardstick for GVNS. The two solvers behave very differently

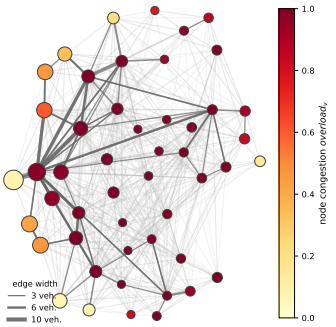


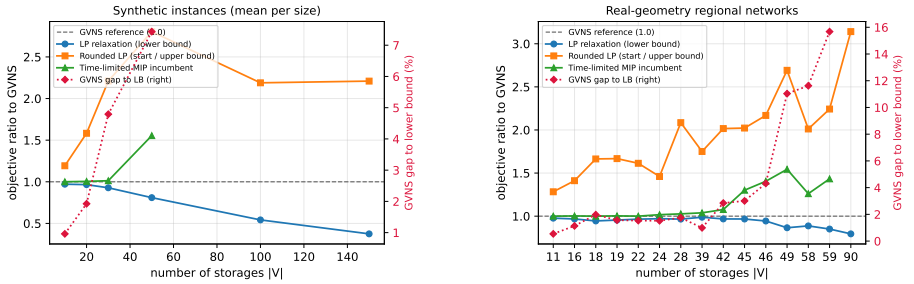

**Fig. 1:** A GVNS solution on the Germany real-geometry network ($|V| = 49$), drawn on its geographic layout. Node area encodes the transshipment capacity $W_v$ and node colour the congestion $overload_v$ (dark = at capacity). All used edges are drawn faintly; backbone edges carrying at least three vehicles are emphasized with width proportional to the vehicle count.

on this model: up to $|V| = 42$ their dual bounds agree to within 0.6% (mean 0.18%), mutually cross-validating the lower bound, but HiGHS finds markedly better incumbents, while SCIP returns *no* feasible solution and no non-trivial dual bound on *every* instance with $|V| \geq 45$. At $|V| \geq 100$ the arc model exhausts memory for both backends. HiGHS is thus both the stronger exact solver here and the natural in-loop choice.

*Exact-solver parameters.* The results above use each solver at its factory settings apart from the time limit, the MIP-gap target and the thread count; no presolve, cut, heuristic or emphasis parameter was tuned. To check that this does not understate the exact baseline, we re-solved a subset of eight instances ($|V| = 30$–59, where the exact solver is the competitor) under three non-default presets per backend: aggressive primal heuristics, disabled presolve and full parallelism for HiGHS; aggressive heuristics, feasibility emphasis and aggressive separation for SCIP. Tuning does not change the picture: for HiGHS the best preset moves the dual bound by at most 0.04% and the incumbent by at most 0.89%, and for SCIP no preset helps at all—aggressive separation and feasibility emphasis degrade the dual bound by up to 2.3% and the incumbent by up to 48%, and on the six subset instances with $|V| \geq 45$ no preset finds a feasible solution where the defaults find none. The default configuration is thus not understating the exact baseline; the full grid is released as `Results/exact_tuning.csv`.

### 5.2 Results

Table 2 summarises the synthetic and real-geometry results; Figure 2 plots the optimality gap and the reference ratios against instance size. Four consistent trends emerge.




**(a)** Synthetic instances (mean per size). **(b)** Real-geometry regional networks.

**Fig. 2:** Left axis: objective ratio of each reference method to GVNS versus network size (below the line at 1.0 is better than GVNS, above is worse). Right axis (dotted): the GVNS gap to the best valid lower bound.

*First, GVNS is near-optimal on small instances.* Its gap to the MIP dual bound is only 0.96% at $|V| = 10$ and stays around 1–2% up to $|V| \approx 20$, then grows with size (4.8% at $|V| = 30$, 7.4% at $|V| = 50$), reflecting the rapidly expanding path space. Since the dual bound is a valid lower bound, this gap bounds the true distance of GVNS from the optimum rather than comparing it against itself.

*Second, the across-seed spread is small but grows with instance size, and ten seeds are needed to see it.* On the synthetic instances the standard deviation of the objective stays below 0.7% of the mean at every size and below 0.2% up to $|V| = 20$. On the largest real networks it is appreciably higher—2.0% on Germany, 2.5% on Italy, 2.2% on European Russia—and, notably, three seeds badly understate it: on Italy the spread measured over three seeds is 0.4% against 2.5% over ten. A single run is therefore representative on small networks, whereas on capacity-tight instances the best of several runs is worth taking.

*Third, GVNS matches or beats the exact solver at equal time, increasingly so with size.* On small instances the time-limited MIP incumbent essentially ties GVNS ($Ratio_{\mathrm{MIP}} \approx 1.00$ for $|V| \leq 30$); from $|V| = 45$ the exact solver falls behind sharply, returning solutions 1.26–1.54× more expensive than GVNS at $|V| = 49$–59, and 1.555× on average over the five $|V| = 50$ instances—that is, up to 55% more expensive for a comparable wall-clock budget. Tight dual bounds are available up to $|V| = 59$; for $|V| \gtrsim 90$ the arc model reaches millions of variables and the exact branch-and-bound becomes intractable, leaving only the closed-form but loose LP-relaxation bound. Even there GVNS keeps improving on its rounded-LP start by large factors (2.2–3.1×, Table 2).

*Fourth, the real-geometry networks mirror the synthetic ones*: the gap to the dual bound rises from 0.5% (Denmark, $|V| = 11$) to 11–16% (Germany $|V| = 49$, Spain $|V| = 59$), and the exact incumbent degrades from parity to 1.3–1.5× GVNS over the same range, corroborating both the fidelity of the generator and the robustness of GVNS.

**Table 2:** Results on the synthetic instances (top, by network size, $n$ instances each) and on the real-geometry networks (bottom). Metrics are as defined in Section 5: $gap_{\mathrm{LB}}$ is the gap to the best MIP dual bound and the three middle columns are objective ratios to GVNS; $\sigma$ is the across-seed standard deviation as a fraction of the mean, $\bar{t}$ the mean GVNS run time. "†": the arc model has millions of variables and the exact branch-and-bound is intractable, leaving only the closed-form LP bounds.

| Instances | $\lvert V\rvert$ | $n$ | $gap_{\mathrm{LB}}$ | LP-relax. | Rounded LP | MIP-inc. | $\sigma$ |
|---|---|---|---|---|---|---|---|
| *Synthetic (mean per size)* | | | | | | | |
| | 10 | 10 | 0.96% | 0.971 | 1.192 | 1.000 | 0.06% |
| | 20 | 7 | 1.92% | 0.966 | 1.583 | 1.005 | 0.18% |
| | 30 | 7 | 4.79% | 0.928 | 2.210 | 1.013 | 0.68% |
| | 50 | 5 | 7.44% | 0.810 | 2.804 | 1.555 | 0.66% |
| | 100 | 3 | † | 0.542 | 2.189 | † | 0.31% |
| | 150 | 3 | † | 0.375 | 2.210 | † | 0.38% |
| *Real-geometry regional networks* | | | | | | | $\bar{t}$ (s) |
| Denmark | 11 | 1 | 0.5% | 0.977 | 1.283 | 1.000 | 615 |
| Belgium | 16 | 1 | 1.1% | 0.968 | 1.411 | 1.003 | 1292 |
| Netherlands | 18 | 1 | 2.0% | 0.945 | 1.664 | 1.000 | 1943 |
| Hungary | 19 | 1 | 1.6% | 0.955 | 1.668 | 1.003 | 1441 |
| Ohio USA | 22 | 1 | 1.5% | 0.965 | 1.614 | 1.000 | 1429 |
| Georgia USA | 24 | 1 | 1.5% | 0.971 | 1.460 | 1.017 | 2147 |
| Illinois USA | 28 | 1 | 1.7% | 0.966 | 2.085 | 1.027 | 2434 |
| California USA | 39 | 1 | 1.0% | 0.988 | 1.751 | 1.039 | 2706 |
| United Kingdom | 42 | 1 | 2.8% | 0.968 | 2.016 | 1.077 | 3343 |
| Sweden | 45 | 1 | 3.0% | 0.967 | 2.022 | 1.301 | 2812 |
| Italy | 46 | 1 | 4.3% | 0.944 | 2.169 | 1.405 | 2837 |
| Germany | 49 | 1 | 11.0% | 0.866 | 2.692 | 1.544 | 3348 |
| Texas USA | 58 | 1 | 11.6% | 0.887 | 2.012 | 1.261 | 3456 |
| Spain | 59 | 1 | 15.7% | 0.851 | 2.243 | 1.433 | 3986 |
| Europe Russia | 90 | 1 | † | 0.795 | 3.141 | † | 5595 |

### 5.3 Algorithmic components

Each component was assessed by re-running GVNS with it switched off, over five seeds and the same budget, against the full method (Table 3). $N_2$ (vehicle consolidation over partially loaded edges) is the dominant operator: removing it costs 7.1% on average and up to 23.8% on the larger synthetic instances, and 7.6% on Germany, so filling spare vehicle capacity is the main lever on the objective. $N_1$ contributes a smaller but consistent improvement. $N_3$ (detours around congested storages) is *situational*: it is inactive when storages are far from their limit, but becomes valuable exactly when the transshipment capacity binds—on Germany, where almost every storage operates at capacity (Figure 1), disabling it costs 4.0%.

The last row replaces the VND by a single-neighborhood local search over $N_1$, keeping the same shake over $N_1$–$N_3$: this is the basic ("pure") VNS, and the gap to it measures what the *general* part of GVNS is worth. That effect

**Table 3:** Contribution of each algorithmic component: *median* objective increase (best of five seeds) relative to the full GVNS on the same instances, seeds and budget, over a representative subset of synthetic ($|V| = 20$–$100$) and real-geometry ($|V| = 16$–$49$) instances. Medians are used because the across-seed spread is heavy-tailed on the capacity-tight networks; means and maxima are quoted in the text. $\bar{t}/\bar{t}_{\mathrm{GVNS}}$ is the run time relative to GVNS.

| Variant | Synthetic | Real | $\bar{t}/\bar{t}_{\mathrm{GVNS}}$ |
|---|---|---|---|
| *Leave-one-out neighborhood ablation* | | | |
| without $N_1$ (reload-aware shortest paths) | +0.8% | +0.8% | 0.80 |
| without $N_2$ (partially loaded edges) | +3.9% | +1.6% | 0.52 |
| without $N_3$ (congestion detours) | +0.1% | +0.2% | 0.68 |
| *Simplified search scheme* | | | |
| Basic VNS (local search restricted to $N_1$) | +0.3% | +0.2% | 0.63 |

is real but modest, and it grows with instance size. Counting an instance as decided only when the difference exceeds the across-seed spread of both runs, GVNS is better on seven of the sixteen instances, Basic VNS on one, and eight are ties. What changes with size is the margin rather than the outcome: where GVNS wins it leads by 0.2–0.6% at $|V| \leq 28$ but by 1.1–2.0% at $|V| \geq 50$. Basic VNS needs only 0.63 of the GVNS time, so the full VND earns its cost as the path space grows. The components are thus not equally important: $N_2$ dominates, the VND structure matters mainly at scale, and $N_3$ is reserved for capacity-tight instances.

## 6 Conclusion

We addressed the design of logistic transport flows with nonlinear, vehicle-induced edge costs and hard storage transshipment capacities, gave arc-based and path-based MILP models with their relaxation, and proposed a GVNS operating in the path space through three problem-specific neighborhoods.

On 35 synthetic and 15 real-geometry instances of up to 150 storages, GVNS came within about one percent of the best dual bound on small networks, and on the medium ones a time-limited exact solver with a comparable budget returned solutions up to 55% more expensive; its optimality gap grows with size, as the path space explodes. The component study showed that consolidating flow over partially loaded edges ($N_2$) is the dominant operator, that congestion detours ($N_3$) pay off only on capacity-tight instances, and that the full VND is worth 1–2% over a basic VNS on the larger networks and a few tenths of a percent on the smaller ones.

Two extensions stand out. The model treats capacities as static, whereas on real networks both road throughput and reload demand vary with the time of day; moving to a time-expanded graph with period-dependent $C_e(t)$ and $W_v(t)$, in the spirit of time-dependent routing [13], is a natural next step, and the path-

based GVNS carries over directly since its neighborhoods need only a period-aware edge metric. Column-generation pricing could also feed the path pool in place of the precomputed shortest-path pools, and validation on operational data remains the ultimate test.

**Code and data availability.** The implementation, the exact-solver harness and the scripts that regenerate every table and figure of Section 5 are at `https://github.com/aponom84/Optimal-Logistic-Net`; the instances are on Zenodo [25] under CC-BY-4.0.

**Acknowledgements.** The study was implemented in the framework of the Basic Research Program at HSE University (HSE-BR-2025-080). Also, this research was supported in part through computational resources of HPC facilities at HSE University.

## References

1. Alumur, S., Kara, B.Y.: Network hub location problems: the state of the art. Eur. J. Oper. Res. **190**(1), 1–21 (2008)
2. Amous, M., Toumi, S., Jarboui, B., Eddaly, M.: A variable neighborhood search algorithm for the capacitated vehicle routing problem. Comput. Oper. Res. **88**, 234–248 (2017)
3. Barnhart, C., Johnson, E.L., Nemhauser, G.L., Savelsbergh, M.W.P., Vance, P.H.: Branch-and-price: column generation for solving huge integer programs. Oper. Res. **46**(3), 316–329 (1998)
4. Barthélemy, M.: Spatial networks. Phys. Rep. **499**(1–3), 1–101 (2011)
5. Batsyn, M., Ponomarenko, A.: Heuristic for a real-life truck and trailer routing problem. Procedia Comput. Sci. **31**, 778–792 (2014)
6. Batsyn, M., Ponomarenko, A.: Heuristic for site-dependent truck and trailer routing problem with soft and hard time windows and split deliveries. In: Pardalos, P., Pavone, M., Farinella, G.M., Cutello, V. (eds.) MOD 2015. LNCS, vol. 9432, pp. 65–79. Springer, Cham (2015)
7. Crainic, T.G.: Service network design in freight transportation. Eur. J. Oper. Res. **122**(2), 272–288 (2000)
8. Crainic, T.G., Gendreau, M.: Heuristics and metaheuristics for fixed-charge network design. In: Crainic, T.G., Gendron, B., Gendreau, M. (eds.) Network Design with Applications to Transportation and Logistics, pp. 91–138. Springer, Cham (2021)
9. Croxton, K.L., Gendron, B., Magnanti, T.L.: A comparison of mixed-integer programming models for nonconvex piecewise linear cost minimization problems. Manage. Sci. **49**(9), 1268–1273 (2003)
10. Gabaix, X.: Zipf's law for cities: an explanation. Q. J. Econ. **114**(3), 739–767 (1999)
11. Gabrel, V., Knippel, A., Minoux, M.: Exact solution of multicommodity network optimization problems with general step cost functions. Oper. Res. Lett. **25**(1), 15–23 (1999)
12. Gastner, M.T., Newman, M.E.J.: The spatial structure of networks. Eur. Phys. J. B **49**(2), 247–252 (2006)

13. Gendreau, M., Ghiani, G., Guerriero, E.: Time-dependent routing problems: a review. Comput. Oper. Res. **64**, 189–197 (2015)
14. Gendron, B., Crainic, T.G., Frangioni, A.: Multicommodity capacitated network design. In: Sansò, B., Soriano, P. (eds.) Telecommunications Network Planning, pp. 1–19. Kluwer, Boston (1999)
15. Ghamlouche, I., Crainic, T.G., Gendreau, M.: Cycle-based neighbourhoods for fixed-charge capacitated multicommodity network design. Oper. Res. **51**(4), 655–667 (2003)
16. Hansen, P., Mladenović, N.: Variable neighborhood search. In: Handbook of Metaheuristics, 3rd edn., pp. 57–97. Springer, Cham (2019)
17. Hansen, P., Mladenović, N., Todosijević, R., Hanafi, S.: Variable neighborhood search: basics and variants. EURO J. Comput. Optim. **5**(3), 423–454 (2017)
18. Hewitt, M., Nemhauser, G.L., Savelsbergh, M.W.P.: Combining exact and heuristic approaches for the capacitated fixed-charge network flow problem. INFORMS J. Comput. **22**(2), 314–325 (2010)
19. Hojny, C., Besançon, M., Bestuzheva, K., et al.: The SCIP Optimization Suite 10.0. arXiv:2511.18580 (2025)
20. Huangfu, Q., Hall, J.A.J.: Parallelizing the dual revised simplex method. Math. Program. Comput. **10**(1), 119–142 (2018)
21. Karakostas, P., Sifaleras, A., Georgiadis, M.C.: A general variable neighborhood search-based solution approach for the location-inventory routing problem with distribution outsourcing. Comput. Chem. Eng. **126**, 263–279 (2019)
22. Kirkley, A., Barbosa, H., Barthelemy, M., Ghoshal, G.: From networks to optimal higher-order models of complex systems. Nat. Phys. **14**, 250–258 (2018)
23. Kostenetskiy, P., Kozyrev, V., Chulkevich, R., Salikova, M., Mishenin, R.: High-performance computing at HSE University. In: Sokolinsky, L., Zymbler, M., Voevodin, V., Dongarra, J. (eds.) Parallel Computational Technologies, pp. 17–29. Springer, Cham (2026)
24. Kovaleva, V., Ponomarenko, A.: A benchmark generator of realistic logistic transport networks with nonlinear edge costs and transshipment capacities. arXiv:2606.20653 (2026)
25. Kovaleva, V., Ponomarenko, A.: A benchmark of realistic logistic transport networks with nonlinear edge costs and transshipment capacities (dataset). Zenodo (2026). `https://doi.org/10.5281/zenodo.20372456`
26. Masri, H., Krichen, S., Guitouni, A.: A multi-start variable neighborhood search for solving the single path multicommodity flow problem. Appl. Math. Comput. **251**, 132–142 (2015)
27. Mjirda, A., Jarboui, B., Mladenović, J., Wilbaut, C., Hanafi, S.: A general variable neighbourhood search for the multi-product inventory routing problem. IMA J. Manag. Math. **27**(1) (2016)
28. Mladenović, N., Hansen, P.: Variable neighborhood search. Comput. Oper. Res. **24**(11), 1097–1100 (1997)
29. OpenStreetMap. `https://www.openstreetmap.org` (accessed 2026-04-11)
30. Rodríguez-Martín, I., Salazar-González, J.J.: A local branching heuristic for the capacitated fixed-charge network design problem. Comput. Oper. Res. **37**(3), 575–581 (2010)
31. Wilson, A.G.: A statistical theory of spatial distribution models. Transp. Res. **1**(3), 253–269 (1967)